

Software Concurrency, Part 1

One CPU, three jobs. We build a program that does all three badly, then work out exactly why it is bad

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 3, printed pages 59–65.

Lab manual: Lab 04 (timers, polling vs interrupts), Lab 03 Section 3.7 (why interrupts beat polling).

1 What we are actually after this week

Weeks 2 and 3 taught you to control a pin. This week the problem changes shape completely, and the new question is: **how does one CPU do several unrelated things at once?**

It cannot, of course. A single Cortex-M4 core executes one instruction at a time. So the real question is how you *interleave* several activities so convincingly that a human watching the LEDs believes they are simultaneous. And the honest way to learn that is to write the obvious program first, watch it fail, and diagnose the failure precisely.

The plan:

1. The three quantities we are going to trade against each other all chapter: **modularity**, **responsiveness**, and **CPU overhead**.
2. The demo system. Two switches, one RGB LED, three behaviours. Small enough to hold in your head, big enough to break.
3. The starter program, written the obvious way. One loop, everything in it.
4. **The analysis**, which is the actual content of this week. Three separate faults, each with a different cause and a different cure.
5. A first look at the menu of cures, so you know what week 5 is going to do.

Item 4 is the week. If you take one thing away, it should be the ability to look at a polling loop and say precisely how long its worst-case response time is and which line of code caused it.

2 Deep dive 1: Three words we need precise definitions for

The whole chapter is a negotiation between three properties, and casual English will not do. So:

The three quantities

Modularity: how well the program groups related code together and separates unrelated code. High modularity means you can change one feature without reading the others.

Responsiveness: how quickly the system responds to an input event. Lower response time is better. Note this is about *latency*, not throughput.

CPU overhead: the portion of time the CPU spends executing code that does no useful work for the application. Waiting in a loop is pure overhead.

Now, the reason these three matter together is that improving one naively damages another. Making a task shorter improves responsiveness and can hurt modularity if you do it by splitting

a coherent activity in half. Adding a scheduler improves modularity and adds a little overhead. Polling more often improves responsiveness and burns more CPU. There is no move that is free.

The book's own roadmap for the chapter is a chain of improvements, each fixing a problem the previous step exposed: start with a plain program, break it into **tasks**, add **interrupts** to get event-triggered processing, make the tasks **shorter**, hand work off to **hardware peripherals**, then eventually **prioritise** and **preempt**. Each step buys modularity or responsiveness or lower overhead, and we will name which one every time.

3 Deep dive 2: The demo system

Here is the thing we are building. An MCU, two switches, and an RGB LED with current-limiting resistors, exactly the circuit from Week 02 Section 5.

The specification has three behaviours:

- **Switch 1 not pressed:** the LED cycles red, then green, then blue, repeating.
- **Switch 1 pressed:** the LED flashes white (all three colours on) and off, repeatedly, until released.
- **Switch 2 pressed:** whichever of the above is running uses *faster* timing.

Response time, defined for this system

The elapsed time between the user pressing switch 1 and the LED starting to flash white. Shorter is better, and **how we share the processor's time among the activities is the main factor that determines it.**

Notice what makes this a genuine concurrency problem rather than a made-up one. Three activities want the CPU at once: watching the switches, running the white flash, running the colour sequence. And two of them involve *waiting*, which is where all the trouble comes from.

Book board vs your board

The book's setup: switch 1 is B1 on the Nucleo at **PC13**, switch 2 is an external button added at **PC7**, and the RGB LED is external on **PA5**, **PA6**, **PA7** through resistors R1–R3. Both switches are **active low**.

On your STM32F3 Discovery you have four user LEDs on port E and a user button on **PA0** which is **active high**, so `SWITCH_PRESSED` loses its inversion. You can run the whole exercise on the on-board LEDs without wiring anything, using LD3/LD5/LD7 as your three "colours". If you do want a real RGB LED, remember Week 02: one resistor per leg, sized per colour, because red and blue want very different values.

4 Deep dive 3: The starter program

Right. Write it the obvious way. One loop, read the switches, drive the LEDs.

Listing 1: The starter program. Book Listing 3.1, from `Flasher1/main.c`.

```
1 #include <stm32f091xc.h>           /* yours: "stm32f3xx.h" */
2 #include "switch.h"
3 #include "rgb.h"
4 #include "delay.h"
5
6 #define W_DELAY_SLOW    150
7 #define W_DELAY_FAST    70
8 #define RGB_DELAY_SLOW  400
9 #define RGB_DELAY_FAST  100
```

```

10
11 void Flasher(void) {
12     uint32_t w_delay    = W_DELAY_SLOW;
13     uint32_t RGB_delay  = RGB_DELAY_SLOW;
14
15     Init_GPIO_RGB();
16     Init_GPIO_Switches();
17
18     while (1) {
19         if (SWITCH_PRESSED(SW1_POS)) {           /* flash white */
20             Control_RGB_LEDs(1, 1, 1);
21             Delay(w_delay);
22             Control_RGB_LEDs(0, 0, 0);
23             Delay(w_delay);
24         } else {                                  /* sequence R, G, B */
25             Control_RGB_LEDs(1, 0, 0);
26             Delay(RGB_delay);
27             Control_RGB_LEDs(0, 1, 0);
28             Delay(RGB_delay);
29             Control_RGB_LEDs(0, 0, 1);
30             Delay(RGB_delay);
31         }
32
33         if (SWITCH_PRESSED(SW2_POS)) {           /* pick the timing */
34             w_delay    = W_DELAY_FAST;
35             RGB_delay  = RGB_DELAY_FAST;
36         } else {
37             w_delay    = W_DELAY_SLOW;
38             RGB_delay  = RGB_DELAY_SLOW;
39         }
40     }
41 }

```

Read it and be honest: it is a perfectly reasonable first program. It is clear, it is short, it obviously implements the specification. If you handed this in it would work on the bench.

The support code is worth a look too, because it is all Week 03 material:

Listing 2: Switch support. Note the inversion in SWITCH_PRESSED. Book Listing 3.2.

```

1 #define SW_GPIO   (GPIOC)           /* both switches live on port C */
2 #define SW1_POS   (13)              /* PC13, the Nucleo user button B1 */
3 #define SW2_POS   (7)               /* PC7, external button */
4 #define MASK(x)   (1UL << (x))
5
6 /* Both switches are active low, so invert the bit we read */
7 #define SWITCH_PRESSED(x)  (!((SW_GPIO)->IDR & MASK(x)))

```

Listing 3: LED control, active low, using BSRR. Book Listing 3.3.

```

1 void Control_RGB_LEDs(int r_on, int g_on, int b_on) {
2     GPIOA->BSRR = (r_on) ? LED_R_ON_MSK : LED_R_OFF_MSK;
3     GPIOA->BSRR = (g_on) ? LED_G_ON_MSK : LED_G_OFF_MSK;
4     GPIOA->BSRR = (b_on) ? LED_B_ON_MSK : LED_B_OFF_MSK;
5 }

```

Recall from Week 03 that these LEDs are wired cathode-to-pin, so LED_R_ON_MSK is a BSRR *reset* and LED_R_OFF_MSK is a BSRR *set*. The names read backwards from the register operation on purpose, so the calling code reads forwards.

And the delay, which is where the bodies are buried:

Listing 4: The busy-wait delay. Two nested loops that do nothing at all.

```

1 void Delay(volatile unsigned int time_del) {
2     volatile int n;

```

```

3 |     while (time_del--) {
4 |         n = 1000;
5 |         while (n--)
6 |             ;
7 |     }
8 | }

```

Beware the trap

Notice the `volatile` on both the parameter and `n`. Without it, the compiler correctly observes that this function computes nothing, has no side effects, and can be deleted entirely. At -O2 your delay would vanish and your LEDs would blur into a single colour.

This is also the reason a busy-wait delay is untrustworthy for real timing: its duration depends on the compiler, the optimisation level, the flash wait states, and the clock speed. Change any one of those and every delay in your program shifts. Week 12 replaces this with a hardware timer, which is accurate because it counts a clock rather than counting instructions.

5 Deep dive 4: The analysis, which is the whole point

Now run it and watch. Here is what actually happens when you press switch 1 while the green LED is lit.

Nothing. The green LED finishes its 400-unit delay. Then blue lights and finishes *its* 400-unit delay. Only then does control return to the top of the loop, the `if` re-evaluates, and the white flashing finally starts.

You waited through the remainder of green plus all of blue. And that is the good case.

5.1 Fault 1: sluggish response

The program only tests the switch **between complete colour cycles**. So the worst-case response time is essentially the full duration of the longest path through the loop body.

Let's put numbers on it. Say one unit of Delay is 1 ms, so `RGB_DELAY_SLOW` of 400 is 400 ms.

Situation	Wait	Why
Press just before the <code>if</code>	≈ 0 ms	Best case, you got lucky
Press just as red lights	≈ 1200 ms	Must finish red, green, and blue
Press just as green lights	≈ 800 ms	Must finish green and blue
Worst case	≈ 1200 ms	3×400 ms

Recall the distinction from Week 01: responsiveness is about *worst-case* latency, not average. The average here is around 600 ms, which sounds tolerable, and the worst case is 1.2 seconds, which for a button feels broken.

Notice also the asymmetry the book points out. **Releasing** the switch responds much faster than pressing it, because the white-flash branch is short: two delays of 150 rather than three of 400. Same program, same switch, response time differing by a factor of four depending on which direction you moved it. That is what a badly shared CPU feels like from the outside.

5.2 Fault 2: lost input events

This one is worse than slow, because slow is merely annoying and this is incorrect.

Press the switch briefly during the colour cycle and release it before the cycle ends. The program never notices. By the time execution reaches the `if`, the switch is back to unpressed, and the entire event has been **silently discarded**.

Polling, and its structural flaw

Polling is repeatedly checking whether something needs attention. Its flaw is that it only samples the world at the moments you happen to look, so **any event shorter than your polling interval can be missed entirely**. The longer your loop body, the wider the blind spots.

And note the vicious feedback here. The longer the code takes to run, the slower the cycle, the wider the blind window, the more events you lose. Adding features makes the loop longer, which makes input handling worse. The program gets less reliable as it grows, which is exactly the wrong direction.

Beware the trap

People try to fix this by polling more often: sprinkle extra `if (SWITCH_PRESSED(...))` checks between the colour changes. It does narrow the window, and it is the wrong fix, for two reasons.

First, it does not close the window, it only shrinks it, so brief-enough events still vanish. Second, and worse, it tangles the switch-reading logic into the middle of the LED-sequencing logic, which destroys the modularity we have not even built yet. You end up with a program that is both still buggy and now unreadable.

The correct fix is to stop polling and let the hardware tell you, which is what interrupts are, and which is week 5 and week 7.

5.3 Fault 3: it is spaghetti

The third fault has nothing to do with timing. Look at `Flasher` again and count how many unrelated responsibilities live in that one function: reading switches, deciding the mode, sequencing three colours, flashing white, and selecting timing constants. Five concerns, one function, all interleaved.

Spaghetti code

Code that is poorly structured because it entangles unrelated features, complicating development and maintenance. The name is about the difficulty of pulling out one strand without disturbing the rest.

Right now the function is thirty lines and you can still read it. That is the trap: spaghetti is not a problem when the program is small, it is a problem when the program grows, and by then it is expensive to fix. Ask yourself the maintenance questions:

- Add a fourth behaviour, say a slow fade. Where does it go? Inside the existing `if`? A new `else if`? Now three branches share the delay variables.
- Change the colour sequence to R, G, B, Y. Which lines do you touch, and are you certain none of them affect the switch logic?
- Reuse the colour sequencer in another project. You cannot, it does not exist as a separable thing.

In the lab

This is precisely the structure your lab manual warns about in Section 3.2.1 of its General Background chapter, on modularisation and drivers. It says that in an STM32F3-based self-balancing robot the subsystems (motors, sensors, filters, communication) interact closely, and that without care to modularise, **unintended interference between components occurs and produces bugs that are hard to isolate.**

That is not generic advice, it is a warning about your final project specifically. A balancing robot has a sensor read, a filter, a PID computation, a motor update, and a telemetry link, all wanting the CPU, all with deadlines. If you build it as one loop with delays in it, you will get exactly the three faults above, and the third one will make the first two impossible to find.

5.4 The verdict

Three faults, and it is worth being precise that they have three different causes, because they need three different cures:

Fault	Root cause	Cure, and when we cover it
Sluggish response	Long loop body, polling	Shorter tasks (wk 5), interrupts (wk 5, 7)
Lost events	Polling samples discretely	Interrupts, event triggering (wk 5, 7)
Spaghetti structure	No separation of concerns	Tasks and a scheduler (wk 5)

Summarised in the book's own words: the system is sluggish and inefficient, may ignore brief inputs, and is structured badly.

6 Deep dive 5: The menu of cures, previewed

So that you know where this is going, here is the full list of moves available, with what each one actually buys. We build the first few next week.

- **Split into tasks.** Buys *modularity*. Buys no responsiveness at all, and in fact costs you a little, because the scheduler now has function-call overhead. Worth doing anyway, because everything else gets easier afterwards.
- **Use interrupts for event triggering.** Buys *responsiveness* and eliminates *lost events*, because the hardware notices the edge whether or not your code was looking. This is the big one.
- **Restructure tasks to finish sooner.** Buys *responsiveness*, because a task that returns quickly lets the next one start sooner. The technique is a finite state machine: instead of "do step 1, wait, do step 2, wait, do step 3", the task does one step per call and remembers where it was.
- **Hand work to hardware peripherals.** Buys *responsiveness and lower overhead simultaneously*, which makes it the best deal on the list. If a timer generates the LED blink, the CPU is not involved at all.
- **Prioritise tasks.** Buys responsiveness for the tasks you care about, at the expense of the ones you do not.
- **Let tasks preempt each other.** Buys the most responsiveness and introduces the hardest bugs, because now a task can be interrupted halfway through modifying a shared variable. Week 8 is entirely about that danger.

Polling vs event triggering

Polling: your code explicitly checks whether processing is needed. The CPU asks, repeatedly, and pays for every question.

Event triggering: the hardware notifies the processor that a specific event has occurred and processing is needed. The CPU is told, once, exactly when it matters.

Everything good about interrupts follows from that inversion.

In the lab

Your Lab 04 is built around exactly this comparison, and now you can see why it is structured that way. Task 1 asks you to blink an LED using a timer with **polling**, checking the timer's update flag in a loop. Task 2 asks you to do the same job with a **timer interrupt**.

The lab will work either way, and the point is not that one produces a nicer blink. The point is what your `while(1)` loop is free to do in each version. In the polling version it is trapped watching a flag. In the interrupt version it is empty, and you could put anything in it.

Your manual's Lab 03 Section 3.7.1 lists the same three benefits: reduced CPU usage, increased responsiveness, and better power efficiency because the chip can sleep and be woken by the interrupt. That third one only exists in the interrupt version, and it is the one that matters if your project ever runs on a battery.

And that is a wrap on the diagnosis. Next week we operate.

7 Tying it back

We asked how one CPU does several things at once, and we answered it by writing the program that does not.

The starter program fails three ways. It is **sluggish**, with a worst-case response time equal to its longest loop path, roughly 1.2 seconds in our numbers, and asymmetric between press and release because the two branches are different lengths. It **loses events** shorter than its polling interval, and gets worse at this as you add features, because features lengthen the loop. And it is **spaghetti**, mixing five concerns in one function, which will not hurt today and will hurt a great deal when it is your self-balancing robot.

Each fault has its own cause, and that is the useful insight. Sluggishness comes from long tasks. Lost events come from polling itself, and no amount of polling harder fully fixes it. Bad structure comes from having no notion of a task at all. So next week we introduce tasks and a scheduler to fix the structure, then interrupts to fix the polling, then finite state machines to shorten the tasks, and finally we start handing the work to hardware so the CPU stops doing it altogether.

And along the way, that `Delay` function with its `volatile int n` is going to look worse and worse.

Cheat sheet: Week 04

The three quantities

Modularity: how well related code is grouped and unrelated code separated.

Responsiveness: how fast the system reacts to an input event. *Worst-case* latency, not average.

CPU overhead: fraction of time spent on code doing no useful application work. Waiting is pure overhead.

No move improves all three for free. Name the trade every time.

The demo system

Two switches, one RGB LED. SW1 released → cycle R, G, B. SW1 pressed → flash white. SW2 pressed → faster timing.

Book pins: SW1 on PC13, SW2 on PC7, LEDs on PA5/PA6/PA7, all **active low**.

Your board: user button PA0 **active high**, LEDs on port E.

The three faults of the starter program

1. Sluggish. Switch tested only between full cycles. Worst case = longest path through the loop body. With 400 ms delays, $3 \times 400 = 1200$ ms. Press and release respond at different speeds because the two branches differ in length.

2. Lost events. Any event shorter than the polling interval is silently discarded. Gets *worse* as the program grows, because features lengthen the loop.

3. Spaghetti. Five concerns in one function. Harmless now, expensive later.

Polling vs event triggering

Polling: code asks repeatedly whether work is needed. Pays for every question. Blind between questions.

Event triggering: hardware tells the CPU when something happened. One notification, exactly on time.

Polling harder shrinks the blind window but never closes it, *and* it tangles the code. Not a fix.

The busy-wait delay, and why it is a liability

`volatile` is mandatory or the compiler deletes the loop entirely at -O2.

Duration depends on compiler, optimisation level, flash wait states, and clock speed. Not portable, not accurate, and 100% CPU overhead while it runs. Replaced by hardware timers in week 12.

The cures, and what each one buys

Split into **tasks** → modularity (and a small overhead cost).

Interrupts → responsiveness, and eliminates lost events.

Shorter tasks via **finite state machines** → responsiveness.

Offload to **hardware peripherals** → responsiveness *and* lower overhead. Best deal available.

Prioritisation → responsiveness for chosen tasks, at others' expense.

Preemption → most responsiveness, hardest bugs. See week 8.

Likely exam prompts from this chapter

- Given a polling loop with stated delays, compute the worst-case response time. *Very likely.*
- Explain why a polling loop can miss a brief input, and why polling more often is not a complete fix.
- Define modularity, responsiveness, and CPU overhead, and give one trade-off between two of them.
- Contrast polling with event triggering, and state one advantage of each.
- Why must a software delay loop's variables be declared `volatile`?